CIW

2.1 Substrukturensuche

Molekül G_1 + Fragment F $\Rightarrow$ Abildg $m: F \to G_1$

Gegeben: Molekülgraph G_1, Fragment F

Gesucht: Kann F als Teilstruktur in G_1 vor? Abildg $m: F \to G_1$

→ NP-vollständiges Problem

Ullmann-Algorithmus: Resumee

Vorteile:
- → klare Trennung v. Kompatibilitätsprüfung der Kanten/Knoten vs. Graph-Algorithmus
- → kleine ableitbende Prüfung der Subgraph-Isomorphie
- → erweiterbar auf nicht zusammenhängende Subgraphen & Moleküle

Nachteile:
- → Matrixform resultiert in Einzelsdritten mit $O(|V_1| \cdot |V_2|)$ Laufzeit
- → iterative Modifikation / Überprüfung (Refinement) der gesamten Matrix bei Erweiterung des Mappings ist laufzeit-intensive

⇒ Effizienzsteigerung durch lokale Erweiterung des Mappings?

⇒ Effizienzverlust durch Vergrößerung der Anzahl partieller Mappings in Backtracking?

CIW

VF2-Algorithmus

Idee: Analyse der vollständigen Zuordnungsmatrix nach jedem Zuordnungsschritt ist sehr aufwendig ↯

Viele Zuordnungen sind aufgrund von Nachbarschaften zu bereits zugeordneten Knoten eh nicht möglich ↯

⇒ Entscheide lokal und dann, welche Knoten zugeordnet werden können

⇒ Verwende eine Look-Ahead Strategie, um Zuordnungen, die nicht vervollständigt werden können, frühzeitig zu erkennen

Input: Graph $G_1 = (V_1, E_1)$, Subgraph $G_2 = (V_2, E_2)$

↳ Alg charakterisiert über Zustände S

$M(s)$: Matching in Zustand s, $M(s) \subseteq V_1 \times V_2$

$P(s)$: Zuordnungen, um die $M(s)$ erweitert werden kann, $P(s) \subseteq V_1 \times V_2$

```
VF2-Match (state s)
  if ({x ∈ V2 | (•,x) ∈ M(s)} == V2)   // alle Knoten aus V2 zugeordnet
    output M(s)
  else
    P(s) = VF2-Calculate-P(M(s), G1, G2)   // Kandidatenmenge
    for (each (x,y) ∈ P(s))
      if (VF2-FEASIBLE (M(s), x, y))   // gültige Erweiterung?
        s' = CREATE-STATE (M(s) ∪ (x,y))
        VF2-Match (s')
        Restore-State (s)   // Entferne Match (x,y) aus DS
```

VF2-Alg: Kandidatenmenge P(s)

Idee: VF2-Calculate-P:

→ erweitere Matching $M(s)$ lokal, d.h. durch ein Paar von Knoten, die adjazent zu bereits zugeordneten Knoten liegen

→ Matching-Reihenfolge: definiere eine willkürliche, aber feste Ordnung $<$ auf der Knotenmenge V_2:

↳ Randbedingung an $<$: Sei $v_1, \dots, v_{|V_2|}$ die durch $<$ definierte Ordnung, dann muss für alle $k \in \{1, \dots, |V_2|\}$ gelten: $v_1, \dots, v_k$ ist zusammenhängend $\Rightarrow$ $<$ durch DFS or BFS

↳ Matches $(\cdot, v)$ werden nur zu $M(s)$ hinzugefügt, wenn für alle $(\cdot, x) \in M(s)$ gilt: $x < v$.

↳ Erweiterung erfolgt strikt durch $<$

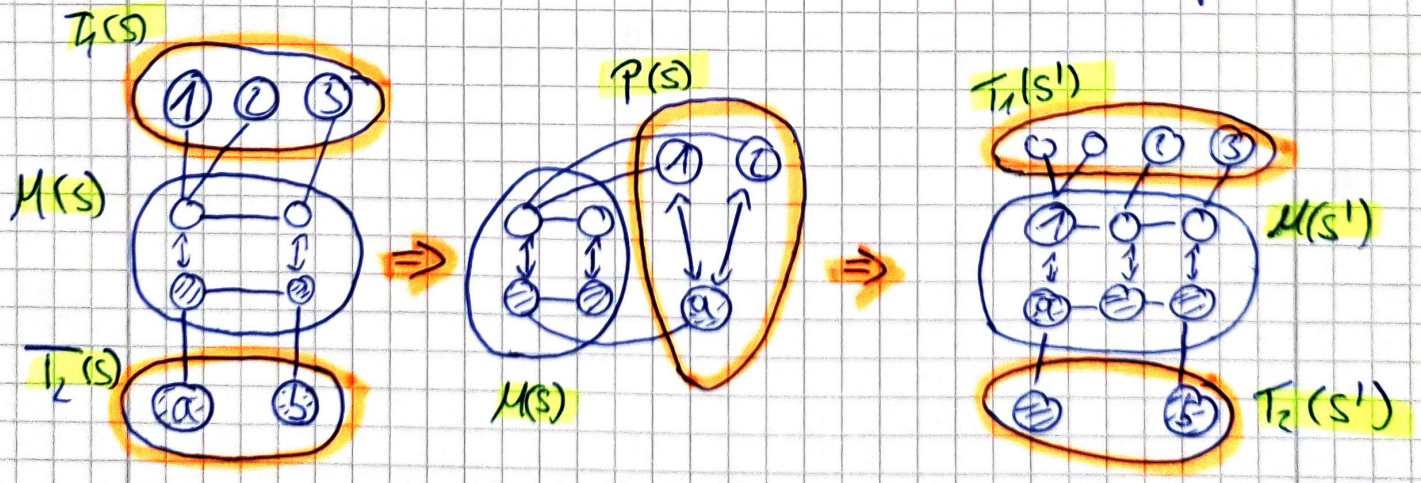
$G_1(s) = (M_1(s), E_1(s))$: Zugeordneter Teilgraph von G_1 in Zustand s

$G_2(s) = (M_2(s), E_2(s))$: Zugeordneter Teilgraph von G_2 in Zustand s

$$T_1(s) = \{v \in V_1 \mid \exists x \in M_1(s) : (x, v) \in E_1\} \setminus M_1(s)$$

$$T_2(s) = \{v \in V_2 \mid \exists x \in M_2(s) : (x, v) \in E_2\} \setminus M_2(s)$$

$$\hookrightarrow P(s) = \{(u, v) \in T_1(s) \times T_2(s) \mid v \text{ minimal bzgl. } <\}$$



CIW

VF2-Alg: Zustände und Datenstrukturen

→ Erster Zustand?

→ Was passiert bei nicht-zshgd Graphen G_2?

↳ $T_1(s)$ & $T_2(s)$ sind leer

↳ (Re-)Initialisierung ist $T_1(s) = V_1 \setminus M_1(s)$, $T_2(s) = V_2 \setminus M_2(s)$

Achtung: zu entfernde Teilgraphen müssen die zu $M_1(s)$ adjazenten Knoten aus $T_1(s)$ entfernt werden!

$$T_1(s) = V_1 \setminus M_1(s) \setminus \{n \in V_1 \mid \exists x \in M_1(s) : (x,n) \in E_1\}$$

→ Repräsentation:

→ Mapping:

$v_i \longrightarrow v_x$

core1[i] = x , core2[x] = i : Knoten $v_i \in V_1$ auf $v_x \in V_2$

→ Nachbarschaftsmengen:

out1[i] = k : ist $k > 0$, dann gilt für Knoten $v_i \in V_1$:

$v_i \in M_1(s) \cup T_1(s)$. k ist die Rekursionstiefe zu der v_i in die Menge $M_1(s) \cup T_1(s)$ aufgenommen wurde

out2[i] = k : analog für Knoten aus V_2

↳ Vektoren müssen nur einmal existieren und können während des Backtrackings auf- und abgebaut werden → $\mathcal{O}(n)$ Speicher

CIW

VF2-Algorithmus: Feasibility-Tests

Der Feasibility-Test prüft, ob ein partielles Matching (Zustand s erweitert um Match (u_1, u_2)) nicht mehr zu einem vollständigen Matching der Substruktur ausgebaut werden kann

$Adj(v) = \{ u \in V \mid (v,u) \in E \}$ // zu v adjazente Knoten

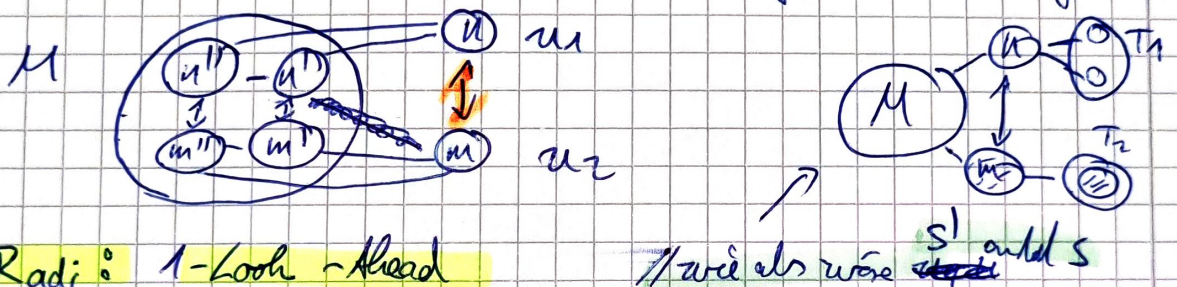
$N_1(s) = V_1 \setminus M_1(s) \setminus T_1(s)$ // Knoten, die nicht zugeordnet und nicht zu zugeordneten Knoten adjazent sind

▶ R_{match}: Konsistenz des erweiterten Matchings

Ziel: $M(s) \cup \{(u_1, u_2)\}$ ist ein valides Subgraph-Matching

$R_{match}(s,(u_1,u_2)) = (\forall v \in M_1(s) \wedge Adj(u_1)\ \exists w \in Adj(u_2) : (v,w) \in M(s))$

$\wedge\ (\forall w \in M_2(s) \wedge Adj(u_2)\ \exists v \in Adj(u_1) : (v,w) \in M(s))$



▶ R_{adj}: 1-Look-Ahead

Ziel: Für Knoten der Nachbarschaftsmenge stehen ausreichend Matching-Partner zur Verfügung

$R_{adj}(s,(u_1,u_2)) = (|Adj(u_1) \cap T_1(s)| \geq |Adj(u_2) \cap T_2(s)|)$

▶ R_{next}: 2-Look-Ahead

Für alle weiterführenden, benachbarten Knoten stehen ausreichend Matching-Partner zur Verfügung

Ziel: ~~Für Knoten der Nachbarschaftsmenge stehen ausreichend Matching-Partner zur Verfügung~~

$R_{next}(s,(u_1,u_2)) = (|Adj(u_1) \cap N_1(s)| \geq |Adj(u_2) \cap N_2(s)|)$

▶ R_{sem}: Kompatibilität der Knoten / Kantenlabel

$\in : E_1 \times E_2 \to \{0,1\}$ Kompatibilität
$\cong : V_1 \times V_2 \to \{0,1\}$ prüfen

$R_{sem}(s,(u_1,u_2)) = (u_1 \cong u_2)$

$\wedge\ (\forall (v_1,v_2) \in M(s), (u_1,v_1) \in E_1 \Rightarrow (u_1,v_1) \in (u_2,v_2))$

Feasibility-Test (VF2-FEASIBLE($M(s), x, y$)):

$F(s,(u_1,u_2)) = R_{match} \wedge R_{adj} \wedge R_{next} \wedge R_{sem}$

BRUNNEN

CIW

VF2 - Alg

Welcher Alg ist schneller? VF2 ~ 10 ms (almost Ever)
Ullman ~ many ≤ 10 ms but also 10 - 1000 ms

Wie wirken sich Rekursionen und explicite Wasserstoffe aus? VF2 (H implizit) < 10 ms
Ullman (H implizit) < 1000 ms VF2 (ohne Reh) > VF2 (ohne Reh) VF2 (H explizit) > 10 ms
Ullman (H explizit) > 1000 ms Ullman (Reh) ≈ 10 * Ullman (ohne Reh)

Wie wirken sich Molekülgröße zu Musstergröße aus?
Größe Ea ziemlich konstant auch bei abweichend H mit/ohne mit Reh / ohne Reh
Ullman etwas zur größe ↯

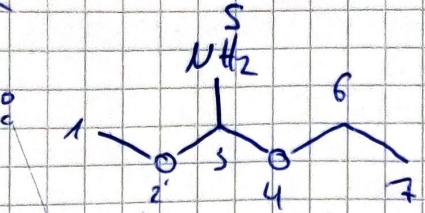
Welchen Einfluss hat die Suche nach dem ersten oder nach allen Treffern?

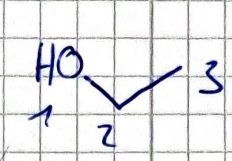
VF2 eine Größenordnung länger 1 -> 10 ms if you want to find alle Treffer
Ullmann ungefähr gleiche Zeit erster Treffer ≈ alle Treffer

VF2 alle treffer linear zur Größe ~~xxx~~, nur erster Treffer constr

CIW - Ex 2

(2-3) VF2

Graph G_1 : 

Subgraph G_2 : 

h = 0

core 2 = [0, 0, 0] , out 2 = [0, 0, 0]

core 1 = [0, 0, 0, 0, 0, 0, 0] , out 1 = [0, 0, 0, 0, 0, 0, 0]

$M = \emptyset$, $T_2 = V_2$, $T_1 = V_1$

$P(S) = \{(1,1), (2,1), (3,1), (4,1), (5,1), (6,1), (7,1)\}$

for each $(x, y) \in P(S)$:

(1,1) -> not feasible : R_{sem} is false due to lable

(2,1) -> $R_{adj} = 1 \wedge R_{match} = 1 \wedge R_{next} = 1 \wedge R_{sem} = 1$ = feasible

(3,1) -> not feasible : R_{sem} is false due to lable

(4,1) -> $R_{adj} = 1 \wedge R_{match} = 1 \wedge R_{next} = 1 \wedge R_{sem} = 1$ = feasible

(5,1) -> not feasible : $R_{sem} = 0$

(6,1) -> not feasible : $R_{sem} = 0$

(7,1) -> not feasible : $R_{sem} = 0$

h = 1

core 2 = [1, 0, 0] ; out 2 = [1, 1, 0]

core 1 = [0, 1, 0, 0, 0, 0, 0] ; out 1 = [1, 1, 1, 0, 0, 0, 0]

$M = \{(1,1)\}$, $T_2 = \{2\}$, $T_1 = \{1, 3\}$

$P(S) = \{(2, 2), (3, 3)\}$, $N_2 = \{3\}$, $N_1 = \{4, 5, 6, 7\}$

for each $(x, y) \in P(S)$:

(2,2) -> $R_{sem} = 1 \wedge R_{adj} = 1 \wedge R_{next} = 0 \wedge R_{match} = 1$

(3,3) -> $R_{sem} = 1 \wedge R_{adj} = 1 \wedge R_{next} = 1 \wedge R_{match} = 1$

BRUNNEN

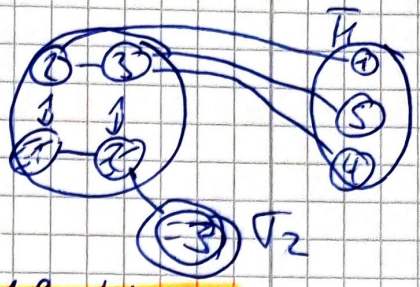
CIW – Ex 2

(2-3) VF2

k = 2

core 2 = [2, 3, 0], out 2 = [1, 1, 2]
core 1 = [0, 1, 2, 0, 0, 0, 0], out 1 = [1, 1, 1, 2, 2, 0, 0]
M = {(1,2), (3,2)}, T_2 = {3}, T_1 = {1, 4, 5}

P(s) = {(1,3), (4,3), (5,3)}



for each (x,y) ∈ P(s):

~~(1,3) R_{sem} = 0 Endpoint~~

(1,3) ⇒ R_{sem} = 1 ∧ adj-1 ∧ R_{next} = 1 ∧ R_{match} = 0

(4,3) → R_{sem} = 0

(5,3) ⇒ R_{sem} = 0

k = 1

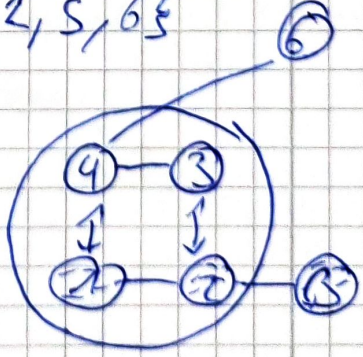
core 2 = [4, 0, 0], out 2 = [1, 1, 0]
core 1 = [0, 0, 0, 1, 0, 0, 0], out 1 = [0, 0, 1, 1, 0, 1, 0]
M = {(4,1)}, T_2 = {2}, T_1 = {3, 6}

(3, 2) → feasible

(6, 2) → feasible

k = 2

core 2 = [4, 3, 0], out 2 = [1, 1, 2]
core 1 = [0, 0, 2, 1, 0, 0, 0], out 1 = [0, 2, 1, 1, 2, 1, 0]
M = {(4,1), (3,2)}, T_2 = {3}, T_1 = {2, 5, 6}
P(s) = {(2, 3), (5, 3), (6, 3)}



(2, 3) → not same label

(5, 3) → not same label

(6, 3) → R_{match} = 0

k = 2

core = [4, 6, 0], out 2 = [1, 1, 2], ~~out 1 = [0, 0, 1, 1, 0, 1, 2]~~
core 1 = [0, 0, 0, 1, 0, 2, 0]
out 1

M = {(4,1), (6,2)}, T_2 = {3}, T_1 = {3, 7}

P = {(3,3), (7,3)}

(3,3) → R_{match}

(7,3) ⇒ Match

M = {(4,1), (6,2), (7,3)}
out 1 = [0, 0, 1, 1, 0, 1, 2]

k = 3

core 2 = [4, 6, 7], out 2 = [1, 1, 2], core 1 = [0, 0, 1, 0, 2, 3]

CIW –

7.2. Definition und Berechnung von Ringbasen

Azyklische Bindung:

wenn der Molekülgraph nach Entfernen der Bindung nicht mehr zshgd ist

Testen der Bindung $e=(x,y)$ in $G=(V,E)$

```
test_cyclic(e, V, E)
E' = E \ {e}
for(each v ∈ V) visited[v] = false
dfs(x, V, E', visited)
if(visited[y]) return 'cyclic'
else return 'acyclic'
```

```
dfs(v, E, visited)
visited[v] = true
for(each w ∈ Adj[v])
  if(not visited[w]) dfs(w, E, visited)
```

$O(|V|+|E|)$ pro test $\Rightarrow O(|E|(|V|+|E|))$

↳ Verbesserung: Ermittlung der Bizusammenhangskomponenten, ein DFS-Durchlauf ist ausreichend $O(|V|+|E|)$

Zerlegung in Ring- und Kettenbindungen:

Gegeben: Molekülgraph (zyklisch) (azyklisch)

Gesucht: Zerlegung in Ring- und Kettenbindungen

↳ Bizusammenhang in ungerichteten Graphen

↳ Ein Knoten v heißt Gelenkknoten, wenn die Entnahme von v den Zshgd zerstört

↳ Zwei Kanten sind bizusammenhängend, wenn es einen einfachen Zyklus gibt, der beide enthält

↳ Bizusammenhang ist eine Äquivalenzrelation auf der Menge der Kanten, die Äquivalenzklassen werden Bi-zshg-Komponenten (BCC) genannt

↳ BCCs lassen sich über DFS bestimmen

↳ BCCs mit nur einer Bindung → azyklische Bindung

Ringbasen

Ziel: Zerlegung von Zusammenhangskomponenten (Ringsystemen)
in eine Menge von Ringen

App: Synthese, Definition Aromatizität, Suche von Molekülstrukturen

1) Wieviele Ringe hat ein Molekül?
2) Zu welchen Ringen gehört ein Atom?

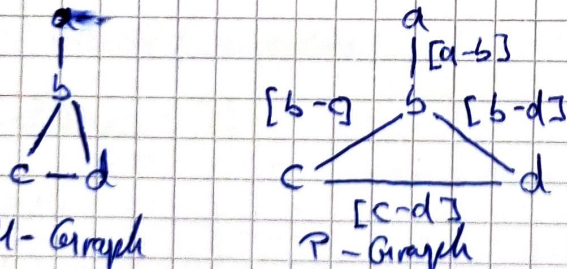
Randbedingungen:
- → chemisch sinnvoll: klar chemischer Ring
- → universell: anwendbar auf jedes Molekül
- → eindeutig: eindeutig aus Molekülstruktur ableitbar
- → polynomiell: # der Ringe sollte polynomiell in der Größe des Moleküls sein
- → effizient berechenbar

$\mathfrak{R}$: Menge aller Ringe

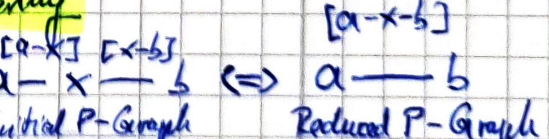
→ lassen sich alle Ringe eines Moleküls aufzählen?

Collapsing P-Graph Alg (Hauser et al)

Idee:
- → Annotiere Pfade an Kanten (P-Graph)
- → Entferne sukzessive Knoten, erzeuge neue Kanten und verbinde die Pfade → ein Multigraph entsteht
- → Schleifen stellen Zyklen dar und können gespeichert werden



Verbindung von Pfaden bei Knotenentfernung



→ Reduktionsschritt (Entfernen eines Knotens)

V.inc: die Inzidenzliste von Knoten v (Menge inzidenter Kanten)

e.to(v): inzidenter Knoten u zu e mit $u \neq v$

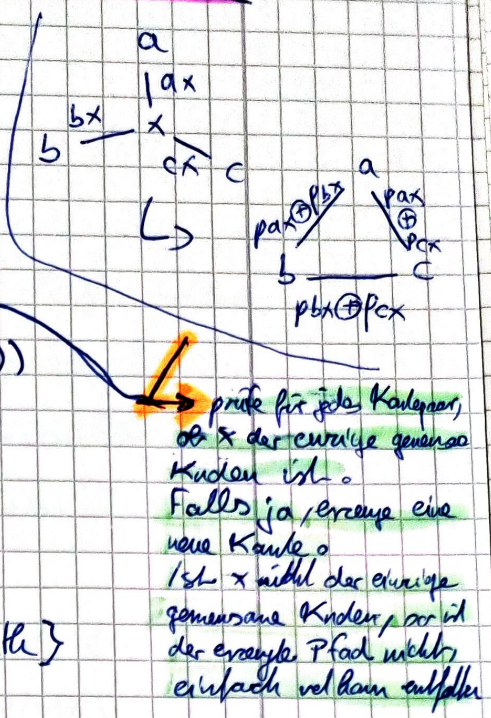
e.path: annotierter Pfad an Kante e,

e.mid: Pfad ohne Endpunkte

BRUNNEN

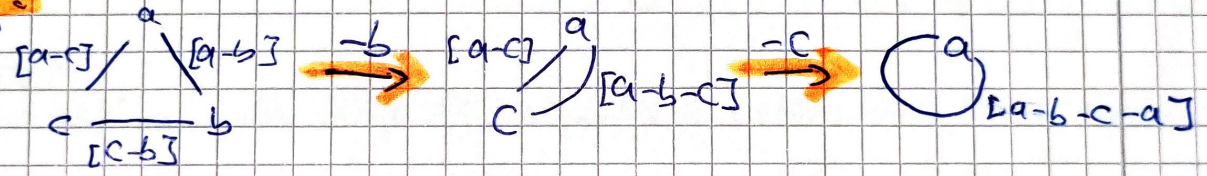
S2: Menge aller Ringe

```
REMOVE-NODE (x, G, R)
  for (each pair e, k ∈ x.inc, k ≠ e)
      if ( e.mid ∧ k.mid == ∅)
          n = new edge (e.to(x), k.to(x))
          n.path = e.path ∪ k.path
          G.E = G.E ∪ {n}
  for (each e ∈ x.inc)
      if (e.path.is-cycle()) R = R ∪ {e.path}
      G.E = G.E \ {e}
  G.V = G.V = {x}
```



prüfe für jedes Kantenpaar, ob x der einzige gemeinsame Knoten ist.
Falls ja, erzeuge eine neue Kante.
Ist x nicht der einzige gemeinsame Knoten, so ist der erzeugte Pfad nicht einfach und kann entfallen

eg:



Heuristische Wahl des nächsten Knotens X

↳ Korrektheit des Alg. ist unabhängig von der Reihenfolge des Entfernens

↳ Effizienzgewinn, wenn jeweils Knoten mit wenigen inzidenten Kanten gelöscht werden

```
ALL-RINGS (G)
  R = ∅
  while ( G.V ≠ ∅)
      x = node from G.V | |x.inc| minimal
      REMOVE-NODE (x, G, R)
  return R
```

CIW

Zyklen (Kreise) und deren Kombination (Addition)

Zyklen, Zyklenaddition und Euklaser

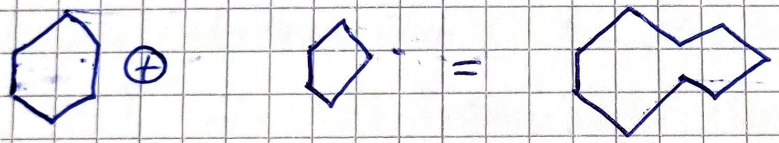
Zyklen:

→ Sei $G = (V, E)$ ein Graph, C ein Teilgraph. C heißt Zyklus, genau dann wenn jeder Knoten in C einen geraden Grad hat. Ein Zyklus C wird beschrieben durch ~~etc~~ die enthaltenen Kanten

→ Ein Zyklus C heißt elementar, genau dann wenn C zshgd ist und jeder Knoten den Grad 2 hat.

→ Seien C, C' zwei Zyklen, die Additionsoperation $\oplus$:

$$C \oplus C' = (C \cup C') \setminus (C \cap C') \quad (\text{XOR})$$



Satz:
Die Additionsoperation ist auf Zyklen abgeschlossen (addition von Zyklen ergibt Zyklen). Sie ist nicht abgeschlossen auf der Menge der elementaren Zyklen.

Überdeckung:
Seien $C = \{C_1, \ldots, C_k\}$ eine Menge von Zyklen auf $G = (V, E)$.

$E[C]$ = Menge der Kanten aller Zyklen, d.h. $E[C] = \bigcup_{c \in C} \bigcup_{e \in c} \{e\}$

↳ C überdeckt G, iff $E[C] = E$ gilt

C(G)

Zyklen (Ringe) & deren Kombination (Addition)

Die Menge $C(G)$ aller Zyklen eines Graphen G wird Zyklenraum (cycles space) genannt.

Eine Menge $\mathcal{B}$ von Zyklen ist Zyklenbasis, if:

1) Jeder Zyklus in G lässt sich durch Addition von Zyklen aus $\mathcal{B}$ konstruieren (vollständige Überdeckung):

$$\forall c \in C(G) : \exists \mathcal{B}_c \subseteq \mathcal{B} : \bigoplus_{b \in \mathcal{B}_c} b = c$$

2) Kein Zyklus aus $\mathcal{B}$ lässt sich durch die anderen Zyklen aus $\mathcal{B}$ konstruieren (lineare Unabhängigkeit der Zyklen in $\mathcal{B}$):

$$\forall c \in \mathcal{B} : \exists \neg\mathcal{B}_c \subseteq \mathcal{B} \setminus \{c\} : \bigoplus_{b \in \mathcal{B}_c} b = c$$

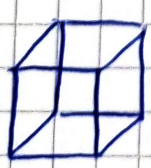
$\oplus$ definiert einen Vektorraum über $C(G)$, jede Basis besteht aus $\mu(G) = |E| - |V| + c(G)$ Zyklen, wobei $c(G) = \#$ der Zshg.-Komponenten darstellt. $\mu(G)$ ist zyklomatische Zahl

↳ $\mu(G)$ = minimale Anzahl von Kanten, deren Entnehmen G in einen Wald überführt (Spannbaum)

Problem: Mehrdeutigkeit bei Ringen gleicher Größe in g symmetrischer Anordnung

zyklomatische Zahl $\overset{!}{=}$ chemische Ringe

eg:



$\mu = 5 \rightarrow$ 6 Ringe

CIW

Minimale Zyklenbasis

Länge von Zyklen:

Sei $C = (V_C, E_C)$ ein Zyklus, dann ist die Länge definiert als $|C| = |E_C|$. Für eine Menge B von Zyklen ist die Länge definiert als $|B| = \sum_{C \in B} |C|$

Minimale Zyklenbasis:

Eine Zyklenbasis B eines Graphen G mit $|B| = \min \{ |B'| \mid B' \text{ ist eine Zyklenbasis von } G \}$ heißt minimale Zyklenbasis

↳ Smallest Set of Smallest Rings (SSSR)

Minimale Zyklenbasen

Eine minimale Zyklenbasis besteht immer aus elementaren Zyklen.

Vorteil: minimale Zyklenbasis besteht somit aus einfachen, relativ kleinen Ringen und ist damit nahe am chemischen Verständnis von Ringen
↳ Algorithmen zur Berechnung minimaler Zyklenbasen können verwendet werden

Satz:
Eine minimale Zyklenbasis ist nicht eindeutig.

Nachteil: Eigenschaft der Eindeutigkeit wird verletzt

CIW

Lineare Abhängigkeit von Zyklen

-> Repräsentation durch Kantenvektoren

$\oplus$ auf Zyklen $\Leftrightarrow$ XOR auf Kantenvektoren

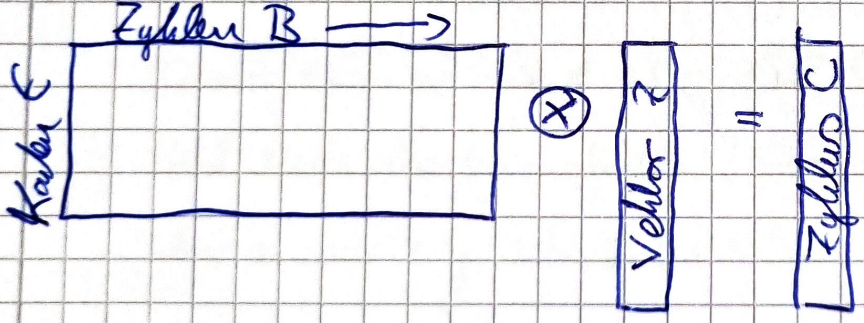
$\otimes$ Multiplikation $\Leftrightarrow$ AND auf Kantenvektoren

Lineare Abhängigkeit von Zyklen:

Sei $\mathcal{B} = \{C_1, \dots, C_k\}$ eine Menge von Zyklen, ein Zyklus C ist linear abhängig, if: Es gibt einen Vektor $z = \{0,1\}^k$ mit

$$C = \bigoplus_{i=1}^{k} (z_i \otimes C_i)$$

$$C[a] = \bigoplus_{i=1}^{k} (z_i \otimes C_i[a])$$



-> Menge B von Zyklen ist linear unabhängig $\Leftrightarrow$ Matrix aus B oder vollen Spaltenrang hat

(Gauß)

CIW

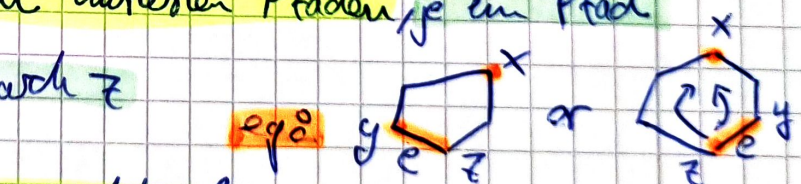
Berechnung minimaler Zyklen

Idee (Horton's Alg):

1) Zähle alle Zyklen auf, die Teil einer minimalen Zyklenbasis sein können (Kandidaten Menge)
2) Betrachte die Zyklen ~~absteigender~~ nach aufsteigender Länge
3) prüfe, ob die Hinzunahme eines Zyklus zu linearer Abhängigkeit führt (Gauß)
4) stoppe, sobald μ Zyklen ausgewählt wurden

Schritt 1: Kandidatenmenge reduzieren

- Horton's Theorem (Strukturaussage „relevanter" Zyklen):
  Sei X ein Knoten in einem Zyklus C einer minimalen Zyklenbasis. Dann existiert ein $e = (y, z) \in C$, sodass C aus e und zwei knotendisjunkten kürzesten Pfaden, je ein Pfad von x nach y und x nach z



- Hortons Kriterium ist notwendig nicht hinreichend
  ↳ Es existieren Zyklen, die Hortons Eigenschaft erfüllen, aber nicht Teil einer minimalen Zyklenbasis sind

→ Bedeutung Hortons Theorem:
Es ermöglicht die Reduzierung einer Initialmenge C_1, aus der eine minimale Zyklenbasis gebildet werden kann. $|C_1| \leq |V||E|$

CIW

Berechnung minimaler Zyklenbasen

Schritt 1: Aufzählen der Initialmenge

→ finde alle Kandidatenkanten von $x \in V$, kürzesten Pfaden $p(x,y)$ und $p(x,z)$ zu einer Kante $e=(y,z) \in E$

→ Entferne alle Kandidaten die nicht einfach sind

→ Reduziere das mehrfach Aufzählen des gleichen Zyklus:

↳ Tiernan Methode: Definiere eine willkürliche Ordnung $<$ auf der Knotenmenge. Kürzeste Pfade $p(x,y)$ für die gilt: $x < u \; \forall u \in p(x,y)$

```
CREATE-INITIAL-CYCLESET(G):
  for (each x ∈ V)
    P_BFS = BFS-SHORTEST-PATHS(G, x)        // all paths
    for (each e = (y,z) ∈ E)
      for (each p(x,y), p(x,z) ∈ P_BFS)
        if (x ≠ min{ p(x,y) ∪ p(x,z) })      continue. // Tiernan-Kri.
        if (p(x,y) ∩ p(x,z) ≠ {x})           continue  // Elementarität
        C_I = C_I ∪ { p(x,y) ∪ (y,z) ∪ p(z,x) }
  return C_I
```

Hartan's Alg:

```
Hartan-Cole-Cycle-Base(G):
  C_I = Create-Initial-Cycleset(G)
  SORT(C_I, Cycle-Length())
  C_B = ∅
  while (|C_B| < μ(G))
    Z = C_I.head(); C_I = C_I.tail()
    if (C_B.is-independent-cycle(Z))
      C_B = C_B ∪ {Z}
```

Aufbau von C_I: $O(|V|^2|E|)$

Sortieren $O(|V||E|)$

while loop $|V||E|$ times:

↳ test auf lineare Unabhängigkeit $O(\mu|E|)$

↳ Total $O(\mu|V||E|^2)$

CIW

Relevante Zyklen

Zyklen einer minimalen Zyklenbasis sind
↳ nahe am chemischen Verständnis von Ringen (Cuban ugelöst)
↳ in polynomieller Zeit berechenbar, aber nicht eindeutig

Relevante Zyklen (Plotkin):
Ein Zyklus ist relevant, iff er zu mindestens einer minimalen Zyklenbasis gehört. C_R ist die Menge der relevanten Zyklen

Vismara Satz:
Ein Zyklus Z ist relevant, iff es keine Menge $C_1, .., C_k$ elementarer Zyklen gibt mit:

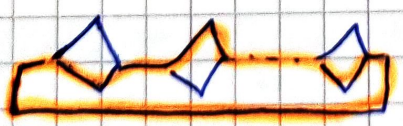
I) $|C_i| < |Z|$, $\forall i \in [1, k]$

II) $Z = C_1 \oplus C_2 \oplus \dots \oplus C_k$

Menge der Relevanten Zyklen ist eindeutig.

Anzahl relevanter Zyklen

p = # relevanter Zyklen , in polynomieller Zeit berechenbar
↳ Worst Case exponentiell in # Kanten



CHW

Berechnung relevanter Zyklen

Vismara's Idee:

-> Einteilung der Relevanten Zyklen in Familien (polynomiell viele)

-> Berechne einen Prototyp für jede Familie (Horton)

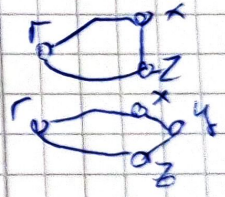
-> Berechne die Größe der Familie (für ρ)

-> Enumeriere die Familie (für C_R)

Vorarbeiten:

Bezeichnet $\prec$ Kantenordnung $\prec$ lässt sich jeder relevante Zyklus Z durch einen Basis-Kanten $\tau(Z)$ und eine Kante $e(Z)=(x,y)$, bzw zwei Kanten $e(Z)=ee'=(x,y,z)$ beschreiben mit:

$$\forall u \in p(\tau,x) \cup p(\tau,z) \cup \{y\} \setminus \{\tau\} : \tau \prec u$$



Relevant Cycle Families (RCF):

Sei C ein relevanter Zyklus, die Familie $F(C)$ ist definiert:

$$F(C) = \{ C' \in C_R \mid |C| = |C'| \wedge \tau(C) = \tau(C') \wedge e(C) = e(C') \}$$

-> RCFs stellen eine (nicht-eindeutige) Partitionierung von C_R dar; die Anzahl der Partitionen $|RCF|$ ist polynomiell in G

-> Ziel: Berechne einen Prototypen-Zyklus für jede RCF. Der Prototyp wird mit RCP (relevant cycle prototype) bezeichnet

CIW

Berechnung Kandidatenmenge

Berechnung Kandidatenmenge C_i'

→ Shortest Path Neighborhood V_r bzgl $\prec$:

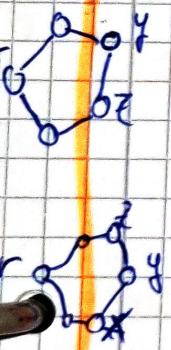
Für einen Knoten r sei $V_r = \{z \in V \mid \exists$ kürzesten Pfad $p(r,z)$ mit:

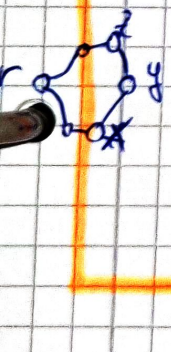
$\forall u \in p(r,z) \setminus \{r\} : r \prec u\}$

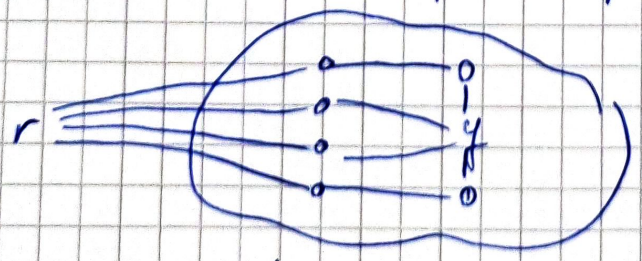
→ V_r kann incl. der kürzesten Pfade mit einer BFS berechnet werden

Visvanna - Variante von Horton's-Alg:

```
CREATE-INITIAL-CYCLESET(G)
  for (each r ∈ V)
    V_r, p, d = BFS-SHORTEST-PATHS(G, r, ≺())
    for (each y ∈ V_r)
      S = ∅                                   // Vorgänger von y entlang kürzester Pfade
      for (each z ∈ V_r ∧ y.Adj)
        if (d(r,z) == d(r,y) - 1)   S = S ∪ {z}
        else if (d(r,z) == d(r,y) and z ≺ y and p(r,y) ∧ p(r,z) == {r})
          C_i' = C_i' ∪ {p(r,y) ∪ p(r,z) ∪ (z,y)}   // ungerade Zyklen
      for (each pair x,z ∈ S)
        if (p(r,x) ∧ p(r,z) = {r})
          C_i' = C_i' ∪ {p(r,x) ∪ p(r,z) ∪ (x,y) ∪ (y,z)}   // gerade Zyklen
```







$d(r,y)-1$ $d(r,y)$

CHW

Berechnung Relevanter Zyklen

Berechnung der RCPs:

-> berechne die Menge der RCPs C_R' parallel zur Erzeugung einer minimalen Cyclenbasis C_E

-> C wird geteilt in die Mengen $C_<$ und $C_=$ von Zyklen, die kürzer bzw. gleich lang wie der aktuell betrachtete Zyklus Z sind

Prüfe:

-> ob Z relevant: $C_< \cup \{Z\}$ ist unabhängig

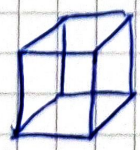
-> ob Z die Basis C erweitert: $C_< \cup C_= \cup \{Z\}$ ist unabhängig

Terminierung:

-> zu jedem relevanten Zyklus Z gibt es eine Basis die Z enthält

-> Sei C eine Zyklenbasis, der längste Zyklus darin habe n Kanten, dann gibt es keinen relevanten Zyklus mit n+1 Kanten

=> Terminiere, sobald C eine Zyklenbasis darstellt und der aktuelle Zyklus Z größer ist als der letzte zu C hinzugefügte Zyklus



$\mu = 5$, $\rho = 6$

BRUNNEN

CTW

Berechnung Relevanter Zyklen

Vismara's Alg:

```
Vismara-Calc-Relevant-Cycle-Prototypes(G1)
  C1' = Create-Initial-Cycleset(G1)
  SORT(C1', Cycle-Length())
  CR' = ∅                          // Prototypen
  C< = ∅ ; C= = ∅                  // Zyklenbasis C< ∪ C=
  prev-size = 0 , max-size = |V|
  while(|C< ∪ C=| < μ(G) or prev-size ≤ max-size)
      z = C1'.head() ; C1' = C1'.tail()
      if(z.size() > prev-size)
          C< = C< ∪ C= , C= = ∅
          prev-size = z.size()
      if(C<.is-independet-cycle(z)) CR' = CR' ∪ {z}   // Relevant
      if((C< ∪ C=).is-independet-cycle(z))            // Part of Basis
          C= = C= ∪ {z}
          if(|C< ∪ C=| == μ(G1)) max-size = z.size()
```

Laufzeit: $O(|V||E|^2) + O(\mu|E|^3)$

↑ Erzeugung von C_1 ↑ Erzeugung von C_R'

Berechnung Relevanter Zyklen

Enumeration relevanter Zyklen aus Prototypen:

→ Berechne zu Wurzelknoten r den gerichteten Graph kürzester Pfade (V_r, E_r) mit: $(y,z) \in E_r$ falls (z,y) auf einem kürzesten Pfad liegt, der keine Knoten u enthält mit $u < r$

→ G_r kann in CREATE-INITIAL-CYCLESET() berechnet werden

→ ALIST-PATHS()-Alg:

Mit Backtracking können in (V_r, E_r) alle kürzesten Pfade enumeriert werden.

→ Sei z ein RCP mit Wurzelknoten r und Kante (y,z):

→ Enumeriere $p(r,y)$ mit ALIST-PATHS()

→ Enumeriere $p(r,z)$ mit ALIST-PATHS()

→ Bilde alle Kombinationen zur Enumeration der zugehörigen RCF

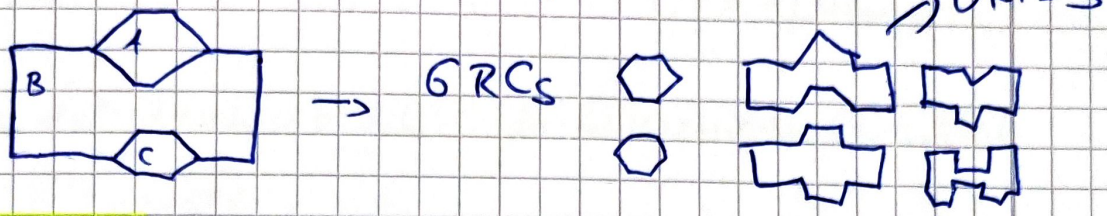
$O(|V||C_R|)$ optimal

CIW

URF
Unique Ring Families URF

Nachteile relevanter Zyklen in Molekülen:

↳ para-substituierte Ringe führen zur Überschätzung der Ringzahl und auch zu chemisch nicht intuitiven Aussagen bzgl. der Anzahl Ringe, zu denen ein Atom gehört:



Essentielle Ringe:

Ein Zyklus $C \in C_R$ heißt essentiell, wenn er in allen minimalen Zyklenbasen enthalten ist

↳ essentielle Zyklen sind polynomiell und eindeutig, beschreiben aber nur ein Teil der Ringstruktur

~~Atom~~ URF-pair-related, URF-related:

Zwei Zyklen C_1, C_2 heißen URF-pair-related, iff:

1) $|C_1| = |C_2|$
2) $C_1 \wedge C_2 \neq \emptyset$
3) $\exists S \subseteq \{z \in \Omega \mid |z| < |C_1|\} : C_1 \oplus (\oplus_{z \in S} z) = C_2$

URF-related ist der transitive Abschluss der Relation URF-pair-related

→URF-pair-related: Zwei Zyklen sind gleich groß, haben gemeinsame Kanten und lassen sich durch Addition kleinerer Zyklen ineinander überführen

→URF-related: Zu zwei Zyklen B, D gibt es eine Sequenz $C_1, C_2 \ldots, C_k$, so dass jeweils $(B, C_2), (C_2, C_3), \ldots (C_k, D)$ URF-pair-related sind

Überlappungsfamilien

URF-related ist Äquivalenzrelation

URF:

eine URF ist eine Äquivalenzklasse der Relation URF-related.
Mit |URF| wird die Länge der Zyklen der URF berechnet.
Die Anzahl URFs eines Graphen → URF-Nummer

URF-Nummer ist durch die RCF-Zahl nach oben beschränkt
und damit polynomiell

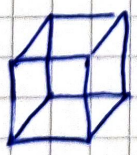
URF berechnen: (Kolodzik): → exponentielle Laufzeit
RCs exponentiell wachsen

→ Berechne RCPs (relevant cycle prototypes)
→ Berechne RCs (Enumeration des RCFs)
→ Evaluiere, welche RCs URF-pair-related sind
→ Berechne, welche RCs URF-related sind (transitiver Abschluss, Berechnung Äquivalenzklassen)
→ Fasse alle Zyklen von URFs in Kantenmengen zusammen

Variante 2 (Kolodzik):

→ Berechne RCPs
→ Evaluiere, welche RCPs URF-related sind
→ Berechne daraus die URF Kantenmenge direkt
↳ $O(|E|^5 + |V|^4)$

Vorteile:
- Schnell < 200 ms
- chemisch sinnvoll
- universell
- evaluierbar
- polynomiell
- effizient berechenbar



URFs = 6
SSSR = 5
RCs = 6

aber: URFs sind im ursprünglichen Sinne keine Zyklen mehr, sondern Kantenmengen, die als ein hierarchische Menge von Zyklen interpretiert werden können

BRUNNEN

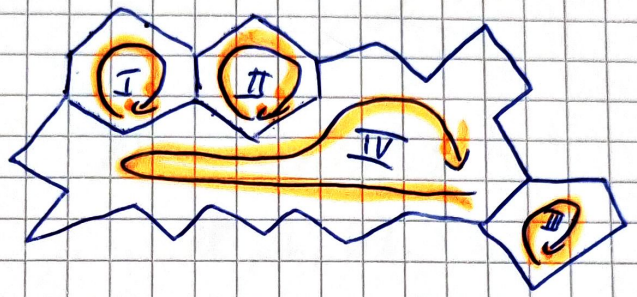
CIW – Ex 3

(3-1) Cycle Bases

1) Vector Bases & Cycle Bases?

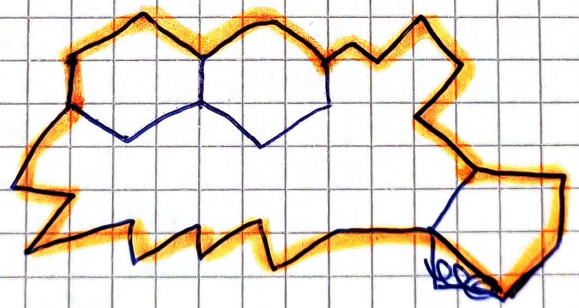
Vector Bases	Cycle Bases
~~Both span~~	
Span a dimension	all cycles in Molekule
(Vectors Add & Mult) linear independent	linear independent (XOR and AND)
$D/M(B)$	$\leq$ $\mu(G)$

2) Minimal Cycle Bases



$C_B = \{I, II, III, IV\}$

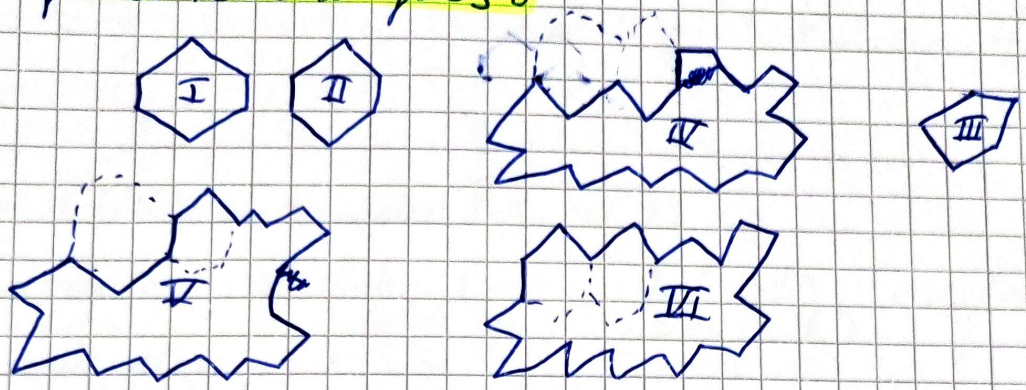
3) $\bigoplus_{c \in C_B} =$



(3-1) Cycle Bases

a)

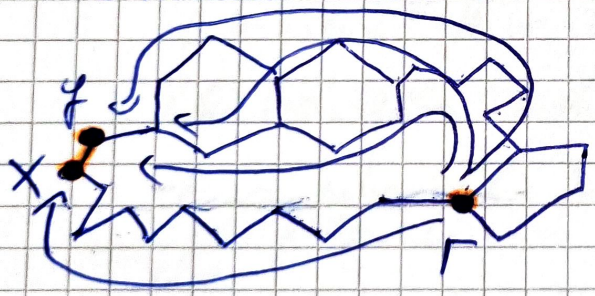
set of all relevant cycles:



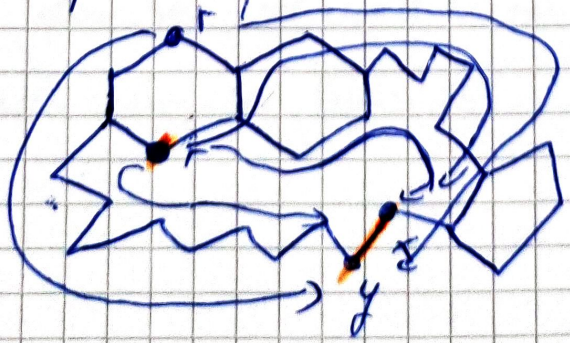
Essential: I, II, III

Non-Essential: IV, V, VI

Atom and bond s.t. all non-essential cycles are part of the same ~~relevant~~ RCF:



Is there a pair of atom and bond s.t. the non-essential cycles are part of more than one RCF?



{IV, V}, {VI}

CIW - Ex3

(3-2)

linker & chains:

A bond is part of a linker iff removing it causes the molecule to split into multiple connected components that contain at least one ring.

moleculeGetDiverseHandles(Mol mol):

```
low ← calculate_low-values(mol)          // number of rings if in ring
cyclic_bonds ← calculate_cyclic_bonds(mol)
cyclic_atom_set ← {}
for bond in cyclic_bonds:
    cyclic_atom_set.insert(bond.first)
    cyclic_atom_set.insert(bond.second)

acyclic_bonds ← for all bonds of mol cyclic, inset.

linker_set ← [ ] , chain_set ← { }

// Start search for linker

for bond in acyclic_bonds:
    if bond.first = H or bond.second = H: continue // ignore H's
    // for both atoms check if there are linker or chain
    if is_linker(mol, bond.first, low, cyclic_atoms)
        linker_set.insert(bond.first)
    else
        chain_set.insert(bond.first)
    if is_linker(mol, bond.second, low, cyclic_atoms)
        linker_set.insert(bond.second)
    else
        chain_set.insert(bond.second)

return cyclic_atom_set, linker_set, chain_set
```

Ole – 63

(57)

```
is_linker_atom(Mol mol, Atom root, Loc, cyclic_atoms):
    if not has_more_than_one_none_h_neighbors(): return false

    vector first_found_ring_atoms
    bool visited

    //Check if every neighbor has at least one ring, for every neighbor counts at most one ring
    ring_in_neighbors <- 0
    for neigh in root->Neighbors():
        has_ring <- false

        visited = [false]   //set all false

        //visit every possible node except root
        visited[root] = true
        dfs_ring_search(neigh, has_ring, visited, root, loc, cyclic_atoms, first_found_ring_atom)   <- check atoms for cyclic and diff locs than root

        if has_ring: ring_in_neighbors ++

    // Check if at least two neighbors are not connected anymore, do this by
      checking first found ring atoms in neighbor subgraph

    zshg_component = 0
    for start in first_found_ring_atoms:
        for end in first_found_ring_atoms:
            for elem in visited: elem <- false

            visited[root] = true
            dfs_find_atom(start, visited)

            if not visited[end]: zshg++

    zshgd = zshgd/2 + 1

    ring_in_neighbors <- connect_ring_counts(ring_in_neighbors)

    return ring_in_neighbors == adjacent_count
```